BRAINSTER

Exercises

Brainster Web Development Academy

Table of contents

TODO Application

General info

- HTML & CSS have already been taken care of
- You'll only need to write JavaScript
- Every exercise builds upon the previous one
- By the end, you get a fully working, client-side ToDo application

Exercise I

1. Define a class called “**ToDo**”
   1.1. The constructor should take a “**title**” input argument
   1.2. The class should also have a “**status**” property, which can be true or false (true for done, false for not done, more on this later).
   1.3. Add **id** value to be unique identifier for each todo

2. Define an array called “**todos**”, which should start empty. It’s purpose is to hold all of the todos which we’ll create.

Exercise II

1. On the form element, add a “**submit**” event listener
   1.1. Prevent the default behavior first (hint: .preventDefault), to avoid reloading the page.

   1.2. Grab the value from the **#todoInput** element, and instantiate a **new TODO()** with the value from the input as the title.

   1.3. Add the new todo to the todos array.

   1.4. Reset the value of the input back to empty (hint: use form.reset() method)

   1.5. To see the result, just log in console the todos array

Exercise III

1. Time to draw the todos on screen
   1.1. Create new <li> element that will contain the item

   1.2. Add label inside <li>, that will hold the title of the todo item

   1.3. Add checkbox inside <li>, that will be used to complete todo item when checked.
   (hint: Remember that checkbox id and label for should be same in order to fire the checkbox change event when label is clicked)

   1.4. Add delete button with classes 'button', 'delete-button'
   1.5. Add edit button with classes 'button', edit-button'
   1.6. Append all this elements inside the created <li>, and append it into the existing ul

Exercise IV

Time to create Mark as Done functionality.

1. Add click event listener to the checkbox (where we created it)
    a. Toggle “done” class to the label

    b. Update data (change the value of item.status)

Exercise V

Time to create Delete functionality.

1. Add click event listener to the delete button (where we created it)
    a. Add confirmation popup
    b. Update ui (remove <li>)
    c. Update data (filter todos array)

Exercise VI

Time to create Edit functionality.

Let’s think this through first, then start writing code.
To update todo item we need to think of ux and ui first.
Questions we should ask ourselves are:

- How will the user update this input?
- What should happen when edit button is clicked?
- Where should the new input appear?
- How will the user confirm the updates?

Only when we have this answers, than we start to code.

1. Add click event listener to the edit button (where we created it)
2. Add boolean value that will keep track if we are editing or we are preparing to edit
3. Use that value in the event listener.
4. if we need to prepare for editing than update ui, create input with item.title value
5. if we need to update then update the ui, and update the todos array

Homework

- Add local storage support.
- Add one more extra functionality. (be creative)